Principal Component Analysis
PCA

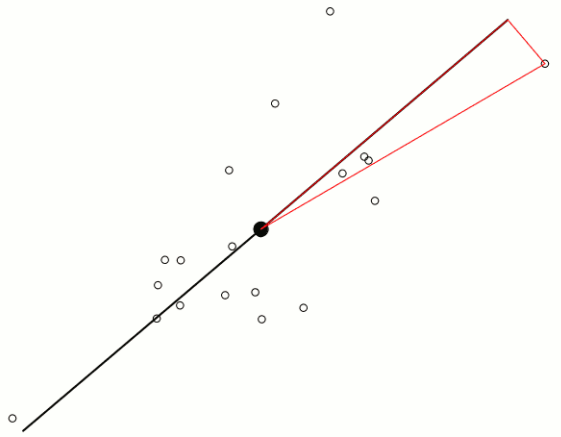
A. prof. Elsayed Mahdy

Principal Component Analysis

A visual + step-by-step lecture with a fully solved numeric example

PCA

Geometric intuition: rotate axes to capture the most variance first



By the end of this lecture, you can:

- Explain PCA as a rotation to new axes that capture maximum variance
- Compute covariance, eigenvalues/eigenvectors, and principal components
- Project data into PC space and interpret explained variance
- Decide how many components to keep (e.g., 90–95% cumulative variance)
- Apply PCA in Python (scikit-learn) and MATLAB

Why PCA is useful

Common problems

- Too many features → slow training + overfitting risk
- Strongly correlated features (multicollinearity)
- High-dimensional data is hard to visualize
- Noise spreads across many dimensions

PCA helps by:

- Finding new axes (PCs) that are uncorrelated
- Keeping only the top k PCs to retain most variance

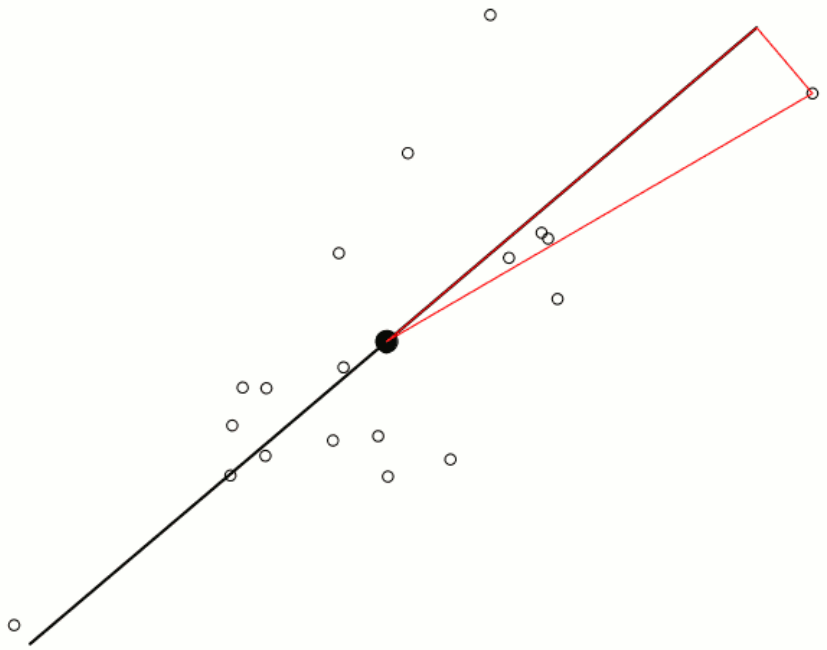
Key idea (one sentence)

PCA finds an orthogonal rotation of the feature space so that the first component captures the maximum variance, the second captures the next maximum (orthogonal to the first), and so on.

$$\textbf{Goal: } \max_{\|w\|=1} \mathrm{Var}(Xw) \quad \text{s.t.} \quad w \perp w_1, \ldots, w_{k-1}$$

PCA is unsupervised (labels are not used).

Geometric intuition (2D)



How to read the picture

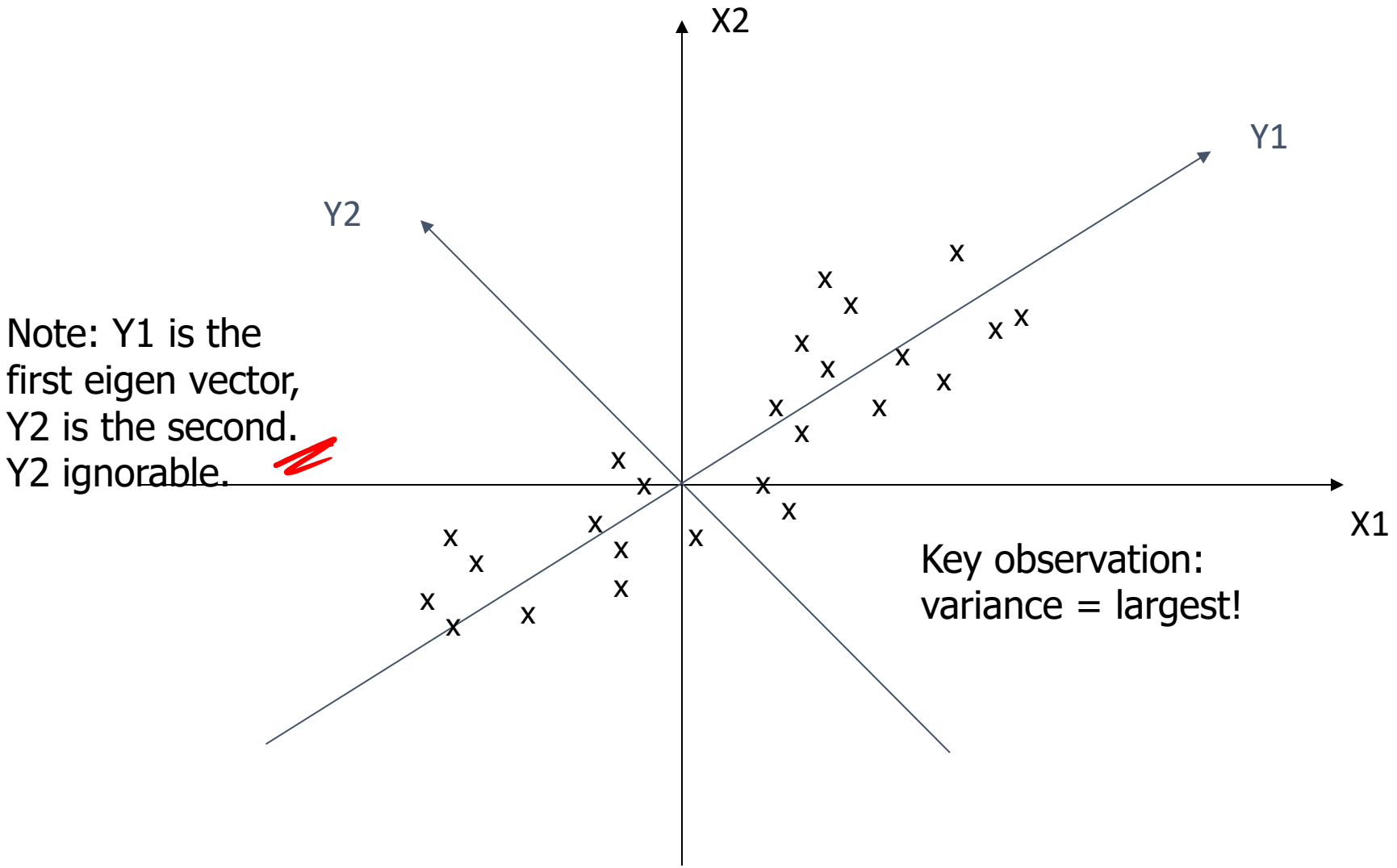
- Black line: direction of maximum spread → PC1
- Orthogonal direction: remaining spread → PC2
- Project each point onto PC1 to get a 1D “score”
- Dropping PC2 keeps most info if its variance is small

In practice: compute PCs from the covariance matrix (or via SVD).

Next: we will compute every step by hand for a small dataset.

- An exploratory technique used to reduce the dimensionality of the data set to 2D or 3D
- Can be used to:
  - Reduce number of dimensions in data
  - Find patterns in high-dimensional data
  - Visualize data of high dimensionality
- Example applications:
  - Face recognition
  - Image compression
  - Gene expression analysis

Principal Components Analysis (PCA)



- Question: how much spread is in the data along the axis? (distance to the mean)
- Variance=Standard deviation^2

$$s^2 = \frac{\sum_{i=1}^{n}(X_i - \overline{X})^2}{(n-1)}$$

Temperature
42
40
24
30
15
18
15
30
15
30
35
30
40
30

Principal Components Analysis (PCA)

Covariance: measures the correlation between X and Y
- cov(X,Y)=0: independent
- Cov(X,Y)>0: move same dir
- Cov(X,Y)<0: move oppo dir

$$\mathrm{cov}(X,Y) = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{(n-1)}$$

X=Temperature	Y=Humidity
40	90
40	90
40	90
30	90
15	70
15	70
15	70
30	90
15	70
30	70
30	70
30	90
40	70
30	90

More than two attributes: covariance matrix

$$C^{nxn} = (c_{ij} \mid c_{ij} = \text{cov}(Dim_i, Dim_j))$$

$$C = \begin{pmatrix} \text{cov}(x,x) & \text{cov}(x,y) & \text{cov}(x,z) \\ \text{cov}(y,x) & \text{cov}(y,y) & \text{cov}(y,z) \\ \text{cov}(z,x) & \text{cov}(z,y) & \text{cov}(z,z) \end{pmatrix}$$

PCA recipe (what you do)

1 **Center the data (subtract the mean).**

Often also standardize to unit variance if features have different units.

2 **Compute the covariance matrix.**

$C = (1/(n-1))\, X^T X$ for mean-centered X.

3 **Eigen-decompose C (or do SVD on X).**

Eigenvectors → principal directions; eigenvalues → variances along PCs.

4 **Sort PCs by decreasing eigenvalue.**

PC1 explains the most variance.

5 **Choose k and project: $Z = X\, W_k$.**

Z are the PC "scores" used for visualization or modeling.

We will now solve steps 1–5 numerically (2 features, 4 points).

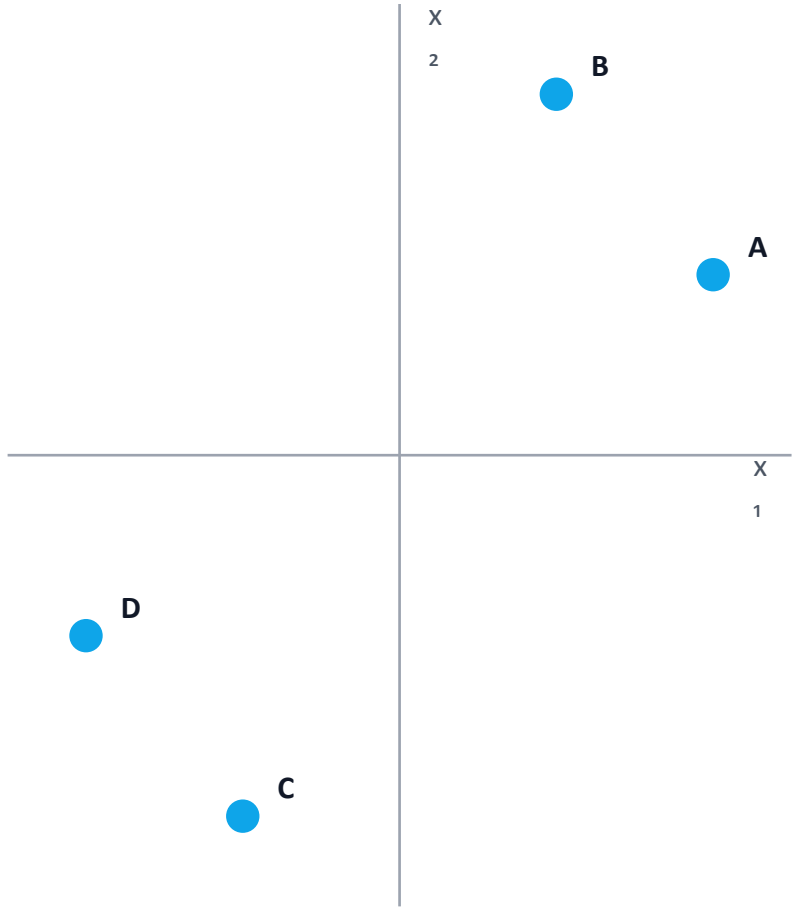
Solved example: start with a tiny dataset

Data (n = 4 points, 2 features)

Point	x_1	x_2
A	2	1
B	1	2
C	-1	-2
D	-2	-1

Mean vector
$\mu = (0, 0)$ → data is already centered
Next: compute covariance $C = (1/(n-1))\, X^T X$

Plot of points (x_1 vs x_2)



These points are correlated: as x_1 increases, x_2 tends to increase.

The PCA Algorithm: 5 Main Steps

1. **Standardize the Data.**
2. **Calculate the Covariance Matrix.**
3. **Compute the Eigenvalues and Eigenvectors of the Covariance Matrix.**
4. **Select the top k Eigenvectors (Principal Components).**
5. **Project the data onto the new subspace.**

These points are correlated: as x_1 increases, x_2 tends to increase.

The PCA Algorithm: Step1

1. **Calculate the Mean:** Find the mean (μ) for each feature (column).
2. **Center the Data:** Subtract the mean from every data point in its respective column.

$$x_{centered} = x - \mu$$

3. **Why?**: PCA is sensitive to scale. Centering ensures the first Principal Component (PC) is not simply the feature with the highest magnitude. The mean of the centered data is now $\mathbf{0}$.

These points are correlated: as x_1 increases, x_2 tends to increase.

The PCA Algorithm: Step2

- The Covariance Matrix ($\mathbf{C}$) is a square matrix that summarizes the statistical relationship between the features.
- **Diagonal Elements:** Variance of each feature ($Cov(X, X)$).
- **Off-Diagonal Elements:** Covariance between features ($Cov(X, Y)$).

$$\mathbf{C} = \begin{pmatrix} \sigma_X^2 & Cov(X,Y) \\ Cov(Y,X) & \sigma_Y^2 \end{pmatrix}$$

- Note: $Cov(X, Y) = Cov(Y, X)$, so the covariance matrix is always **symmetric**.

These points are correlated: as x_1 increases, x_2 tends to increase.

Examples

Example 1
Dataset (4 samples, 2 features)
Let the two features be x_1 and x_2.

$$X = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 4 \\ 5 & 5 \end{bmatrix}$$

We will do PCA using **mean-centering** (no standardization) because the two features are on a similar scale.

Step 1) Compute the mean of each feature

$$\mu = [\mu_{x_1} \quad \mu_{x_2}] = \left[\frac{2+3+4+5}{4} \quad \frac{1+2+4+5}{4}\right] = [3.5 \quad 3]$$

Step 2) Center the data

$$X_c = X - \mu$$

Row-by-row:

- For $(2, 1)$: $(2 - 3.5, 1 - 3) = (-1.5, -2)$
- For $(3, 2)$: $(-0.5, -1)$
- For $(4, 4)$: $(0.5, 1)$
- For $(5, 5)$: $(1.5, 2)$

So:

$$X_c = \begin{bmatrix} -1.5 & -2 \\ -0.5 & -1 \\ 0.5 & 1 \\ 1.5 & 2 \end{bmatrix}$$

These points are correlated: as x_1 increases, x_2 tends to increase.

Examples

Step 3) Compute the covariance matrix

For $n = 4$samples:

$$C = \frac{1}{n-1} X_c^T X_c = \frac{1}{3} X_c^T X_c$$

3.1 Variance of x_1

$$\text{Var}(x_1) = \frac{1}{3}\sum(x_{1i} - \mu_{x_1})^2 = \frac{1}{3}((-1.5)^2 + (-0.5)^2 + (0.5)^2 + (1.5)^2)$$

$$= \frac{1}{3}(2.25 + 0.25 + 0.25 + 2.25) = \frac{5}{3} = 1.6667$$

3.2 Variance of x_2

$$\text{Var}(x_2) = \frac{1}{3}((-2)^2 + (-1)^2 + (1)^2 + (2)^2) = \frac{1}{3}(4 + 1 + 1 + 4) = \frac{10}{3} = 3.3333$$

3.3 Covariance $\text{Cov}(x_1, x_2)$

$$\text{Cov}(x_1, x_2) = \frac{1}{3}\sum(x_{1i} - \mu_{x_1})(x_{2i} - \mu_{x_2})$$

These points are correlated: as x_1 increases, x_2 tends to increase.

Examples

$$= \frac{1}{3}((-1.5)(-2) + (-0.5)(-1) + (0.5)(1) + (1.5)(2))$$
$$= \frac{1}{3}(3 + 0.5 + 0.5 + 3) = \frac{7}{3} = 2.3333$$

So the covariance matrix is:

$$C = \begin{bmatrix} \frac{5}{3} & \frac{7}{3} \\ \frac{7}{3} & \frac{10}{3} \end{bmatrix} \approx \begin{bmatrix} 1.6667 & 2.3333 \\ 2.3333 & 3.3333 \end{bmatrix}$$

Step 4) Find eigenvalues (variance along principal directions)

Solve:

$$\det(C - \lambda I) = 0$$
$$\det\left(\begin{bmatrix} \frac{5}{3} - \lambda & \frac{7}{3} \\ \frac{7}{3} & \frac{10}{3} - \lambda \end{bmatrix}\right) = 0$$
$$\left(\frac{5}{3} - \lambda\right)\left(\frac{10}{3} - \lambda\right) - \left(\frac{7}{3}\right)^2 = 0$$

This simplifies to:

$$9\lambda^2 - 45\lambda + 1 = 0$$

So:

$$\lambda_{1,2} = \frac{15 \pm \sqrt{221}}{6}$$

These points are correlated: as x_1 increases, x_2 tends to increase.

Examples

Numerically:

- $\lambda_1 \approx 4.977678$(largest → **PC1**)
- $\lambda_2 \approx 0.022322$(small → **PC2**)

Explained variance ratio

Total variance $= \lambda_1 + \lambda_2 \approx 5$

$$\text{EVR}_1 = \frac{\lambda_1}{\lambda_1 + \lambda_2} \approx \frac{4.977678}{5} = 0.995536 \ (\approx 99.55\%)$$

$$\text{EVR}_2 \approx 0.004464 \ (\approx 0.45\%)$$

Meaning: **almost all information is captured by PC1.**

Step 5) Find eigenvectors (principal directions)

PC1 eigenvector v_1

Solve $(C - \lambda_1 I)v_1 = 0$. One (unnormalized) solution is:

$$v_1 \propto \begin{bmatrix} \frac{14}{\sqrt{221} + 5} \\ 1 \end{bmatrix} \approx \begin{bmatrix} 0.7047 \\ 1 \end{bmatrix}$$

Normalize to unit length:

These points are correlated: as x_1 increases, x_2 tends to increase.

Examples

$$v_1 = \begin{bmatrix} 0.5760 \\ 0.8174 \end{bmatrix}$$

(Any sign flip $[-v_1]$ *is equivalent.)*

PC2 eigenvector v_2**(orthogonal to** v_1**)**

$$v_2 = \begin{bmatrix} -0.8174 \\ 0.5760 \end{bmatrix}$$

Step 6) Project data onto the principal components (scores)

For each centered sample $x_{c,i}$, the PC1 score is:

$$z_{1,i} = x_{c,i} \cdot v_1$$

Compute each:

1. $x_c = (-1.5, -2)$
   $z_1 = (-1.5)(0.5760) + (-2)(0.8174) = -0.8640 - 1.6348 = -2.4988 \approx -2.4989$

2. $x_c = (-0.5, -1)$
   $z_1 = (-0.5)(0.5760) + (-1)(0.8174) = -0.2880 - 0.8174 = -1.1054$

3. $x_c = (0.5,1)$
   $z_1 = (0.5)(0.5760) + (1)(0.8174) = 0.2880 + 0.8174 = 1.1054$

4. $x_c = (1.5,2)$
   $z_1 = (1.5)(0.5760) + (2)(0.8174) = 0.8640 + 1.6348 = 2.4988 \approx 2.4989$

These points are correlated: as x_1 increases, x_2 tends to increase.

Examples

So, the **1D representation** (keeping only PC1) is:

$$z_1 = \begin{bmatrix} -2.4989 \\ -1.1054 \\ 1.1054 \\ 2.4989 \end{bmatrix}$$

(If you also project onto PC2: $z_{2,i} = x_{c,i} \cdot v_2$, you'll get very small values because λ_2 is tiny.)

Step 7) (Optional but great for teaching) Reconstruct using only PC1

Approximate the original data using just PC1:

$$\hat{x}_i = \mu + z_{1,i} v_1^T$$

Example for the first point $z_1 = -2.4989$:

$$\hat{x} = [3.5 \quad 3] + (-2.4989)[0.5760 \quad 0.8174] = [3.5 - 1.4395 \quad 3 - 2.0426] = [2.0605 \quad 0.9574]$$

Reconstructed points (PC1 only) are approximately:

$$\hat{X} \approx \begin{bmatrix} 2.0605 & 0.9574 \\ 2.8632 & 2.0964 \\ 4.1368 & 3.9036 \\ 4.9395 & 5.0426 \end{bmatrix}$$

These are very close to the original X, showing **why PCA is good for compression/denoising**.

These points are correlated: as x_1 increases, x_2 tends to increase.

Examples

What students should conclude

- **PC1 explains ~99.55%** of variance → reducing from 2D to 1D loses very little information.
- The principal direction is:

$$v_1 = \begin{bmatrix} 0.5760 \\ 0.8174 \end{bmatrix}$$

- The PCA "new coordinate" (feature) for each sample is its score $z_{1,i}$.

These points are correlated: as x_1 increases, x_2 tends to increase.

How to run PCA in practice

Python (scikit-learn)

```python
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# X: (n_samples, n_features)
Xz = StandardScaler().fit_transform(X)

pca = PCA(n_components=0.95)
Z = pca.fit_transform(Xz)

print(pca.n_components_)
print(pca.explained_variance_ratio_)
```

MATLAB

```matlab
% X: n-by-d
Xz = zscore(X);
[coeff, score, latent, ~, explained] = pca(Xz);

% choose k for 95% variance
k = find(cumsum(explained) >= 95, 1);
Z = score(:,1:k);
```

Tip: fit scaler + PCA on training data only, then transform validation/test the same way.

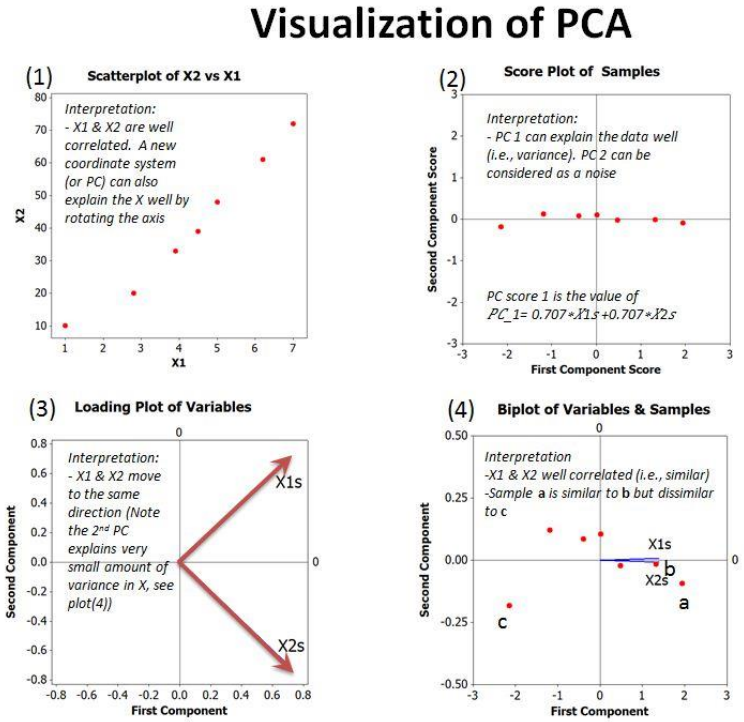
Common pitfalls + quick recap

Pitfalls (teach these!)

- Scaling: if units differ, standardize first (otherwise big-unit features dominate).
- Outliers can rotate PCs сильно (robust scaling may help).
- PCA is linear: curved structure may need kernels / t-SNE / UMAP for visualization.
- Interpretability: PCs are mixtures of original features (look at loadings).

Recap in 10 seconds:
Center → covariance/SVD → eigenvectors = PCs → project → keep top k.

Visual summary



Questions?

الرواد الرقميون
DiGiLiANS

وزارة الاتصالات
وتكنولوجيا المعلومات

THANK you